LIFELINE — Algorithm-Selection Proposal

Group 24 | Iteration 2 decision plan | Candidate algorithms, selected methods and assumptions

Algorithm selection

The selected methods match the workbook's integer-valued inputs and graph structures. Small-tier cases provide exact baselines; larger cases use methods whose runtime is appropriate for the corresponding input size.

Module	Problem (Input, Output & Objective)	Candidates (Time Complexity)	Selection (Primary & Baseline)
A. Relief Inventory	Input: n indivisible crates, capacity W, and crate values. Output: Selected crates with maximum total value without exceeding capacity. Objective: Maximize total value under the truck's weight capacity.	1. Greedy by value-to-weight ratio: O(n log n) 2. Exhaustive search: O(2^n)	Primary: Exhaustive Search Baseline: Greedy selection
B. Route Planning	Input: Graph with V locations, E roads, a source location, and blocked roads. Output: Reachable locations, cheapest route and fewest-roads route. Objective: Determine reachability and suitable routes on today's road network.	1. Dijkstra (binary heap): O((V + E) log V) 2. BFS: O(V + E)	Primary: Dijkstra Baseline: BFS
C. Rescue Scheduling	Input: n rescue windows with start/end times and, where applicable, values. Output: A maximum-size compatible set of windows, or a compatible set with maximum total value. Objective: Schedule the greatest number of non-overlapping windows, or maximize their total value.	1. Greedy earliest-finish-time: O(n log n) 2. Exhaustive search: O(2^n)	Primary: Greedy earliest-finish-time Baseline: Exhaustive Search
D. Emergency Queue	Input: n incidents with urgency levels and release times. Output: Incidents processed in urgency order, one per tick, among those already released. Objective: Select the most urgent available incident at each tick.	1. Heap-based priority queue: O(n log n) total for processing n incidents 2. Insertion Sort: O(n^2)	Primary: Heap-based Priority Queue Baseline: Insertion Sort
E. Radio Logs	Input: Radio logs, a search phrase, and two reports. Output: All phrase start positions (including overlaps), minimum edit distance with an edit script, and a prefix-free binary code for the log. Objective: Search, compare, and compact radio-log data.	1. KMP string matching: O(n + m) 2. Naive string matching: O(nm) 3. Huffman coding: O(k log k)	Primary: KMP for phrase search; Huffman for compression Baseline: Naive String Matching
F. Network Planning	Input: Weighted, undirected graph with V locations and E roads. Output: Roads linking all reachable locations with minimum total cable cost. Objective: Connect the network at minimum total cost.	1. Kruskal: O(E log E) 2. Prim (binary heap): O(E log V)	Primary: Kruskal Baseline: Prim

Module	Problem (Input, Output & Objective)	Candidates (Time Complexity)	Selection (Primary & Baseline)
G. Chains and Cuts	Input: Matrix dimensions for a chain and a drum of length L with prices for available cut lengths. Output: A matrix multiplication bracketing with minimum scalar multiplications and a drum-cutting plan with maximum total price. Objective: Optimize the order of multiplication and the revenue from cutting.	1. Matrix Chain Multiplication DP: O(n^3) 2. Rod Cutting DP: O(L^2) 3. Exhaustive search: exponential time	Primary: Dynamic Programming for both problems Baseline: Exhaustive Search
H. Hard Planning Questions	Input: Safety rules, ration values and target sum, site distances, and a sensor graph. Output: A satisfying safety assignment, exact ration selection, shortest closed tour, and a sensor set of size at most k covering every edge. Objective: Find solutions satisfying all stated constraints.	1. Backtracking: exponential worst case 2. Exhaustive search: exponential worst case	Primary: Backtracking Baseline: Exhaustive Search
I. Nearest Sites and Checksums	Input: n site coordinates and two long integers. Output: The pair of sites with minimum squared Euclidean distance and the product of the two integers. Objective: Find the closest pair efficiently and multiply large numbers.	1. Divide-and-conquer closest pair: O(n log n) 2. Brute-force closest pair: O(n^2) 3. Karatsuba multiplication: $O(n^{\log_2 3})$ 4. Grade-school multiplication: O(n^2)	Primary: Divide-and-Conquer for closest pair; Karatsuba for multiplication Baseline: Brute Force and Grade-School Multiplication

Work division

The following is the proposed ownership split for Iteration 1 and the proposed module allocation for Iteration 2.

Member	Contribution
Muhammad Aun	Project coordinator and team lead. Developed the Python generator and managed overall project coordination and integration.
Maaz Sher	Developed the validator and implemented the workbook validation checks.
Muhammad Bin Shahid	Prepared project documentation and developed `make_tests.py` and `test_suite.py` for validator testing.